EWB CLOUD LABS | AWS ARCHITECT ASSOCIATE

AWS VPC Peering: Private Connectivity & Bastion Architecture

Student Name:	Hamza Hafeel	**AWS Region:**	Asia Pacific (Hyderabad) — ap-south-2
Batch:	Batch 1	**AWS Account ID:**	453732174454
Project Title:	AWS VPC Peering	**Completion Date:**	20 October 2026

01 / ARCHITECTURE, CIDRs & RESOURCE INVENTORY

1.1 Network Topology & CIDR Design

This project establishes a private network connection between two isolated Virtual Private Clouds (VPC A and VPC B) in the ap-south-2 region using an AWS VPC Peering connection (ewb-a-b-peer).

- **Non-Overlapping CIDRs:** VPC A uses 10.10.0.0/16 and VPC B uses 10.20.0.0/16. Because VPC peering requires distinct, non-overlapping IPv4 ranges to route packets unambiguously, each /24 subnet is allocated cleanly from its parent VPC range.
- **Workload Isolation & Bastion Access:** Both application workloads (ewb-app-a at 10.10.2.10 and ewb-app-b at 10.20.1.10) reside in private subnets with no public IPv4 addresses or internet gateways in VPC B. Administrative access is centralized through a single public bastion host (ewb-bastion at 10.10.1.10) in VPC A using OpenSSH ProxyJump.

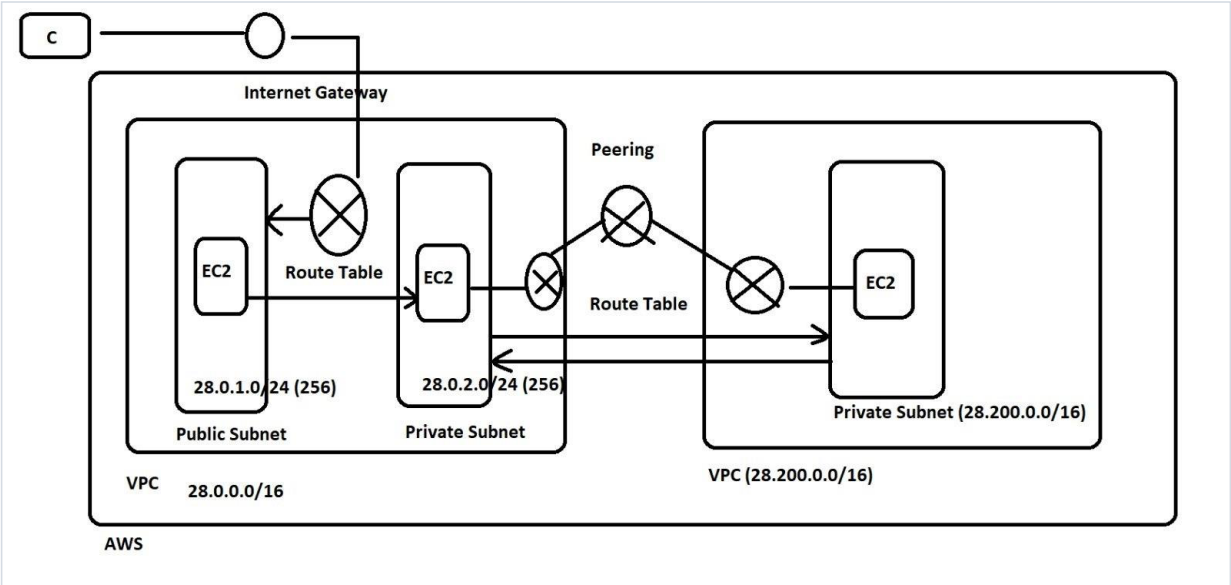


Figure 1: AWS VPC Peering Architecture with Bastion ProxyJump

1.2 Actual AWS Resource Inventory

Below is the complete inventory of deployed AWS resources and their actual IDs recorded from the console:

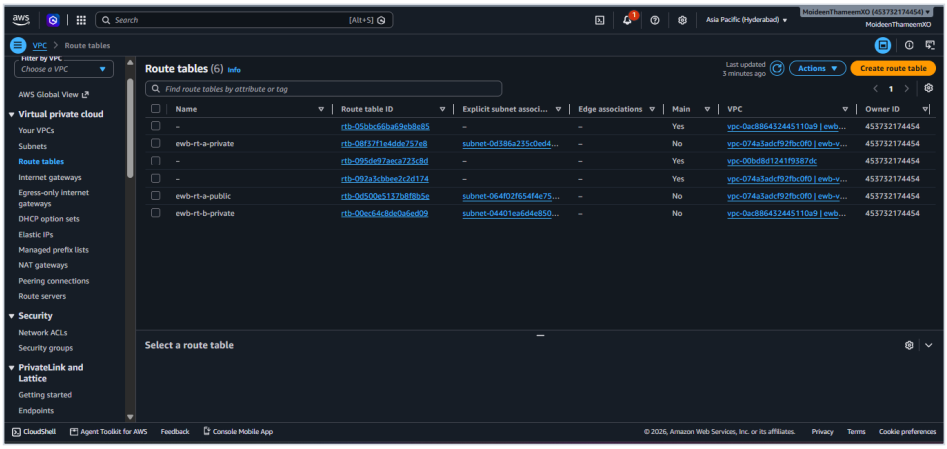
Resource / Name Tag	Resource Type	Actual AWS ID	CIDR / IPv4 Details
ewb-vpc-a	VPC (Requester)	vpc-074a3adcf92fbc0f0	10.10.0.0/16
ewb-vpc-b	VPC (Accepter)	vpc-0ac886432445110a9	10.20.0.0/16
ewb-a-public	Public Subnet (VPC A)	subnet-064f02f654f4e75...	10.10.1.0/24 (ap-south-2b)
ewb-a-private	Private Subnet (VPC A)	subnet-0d386a235c0ed4...	10.10.2.0/24 (ap-south-2b)
ewb-b-private	Private Subnet (VPC B)	subnet-04401ea6d4e850...	10.20.1.0/24 (ap-south-2b)
ewb-igw-a	Internet Gateway	—	Attached to ewb-vpc-a
ewb-a-b-peer	VPC Peering Connection	pcx-03af8378d3fd60f65	Active (ewb-vpc-a <> ewb-vpc-b)
ewb-rt-a-public	Route Table (VPC A Public)	rtb-0d500e5137b8f8b5e	Associated to ewb-a-public
ewb-rt-a-private	Route Table (VPC A Private)	rtb-08f37f1e4dde757e8	Associated to ewb-a-private
ewb-rt-b-private	Route Table (VPC B Private)	rtb-00ec64c8de0a6ed09	Associated to ewb-b-private
ewb-sg-bastion	Security Group (VPC A)	—	Inbound TCP 22 from 49.37.171.106/32
ewb-sg-app-a	Security Group (VPC A)	sg-0a40c302570f61925	TCP 22 (10.10.1.10/32), ICMP (10.20.1.10/32)
ewb-sg-app-b	Security Group (VPC B)	sg-07664b2164498c230	TCP 22 (10.10.1.10/32), ICMP (10.10.2.10/32)
ewb-bastion	EC2 Instance (t3.micro)	i-0d0046a7abdf387f2	Private: 10.10.1.10 | Public: 18.60.91.116
ewb-app-a	EC2 Instance (t3.micro)	i-05c2641e007a3f3de	Private: 10.10.2.10 | Public: None
ewb-app-b	EC2 Instance (t3.micro)	i-0cd9bb4f269a42c92	Private: 10.20.1.10 | Public: None

02 / SUBNETS, INTERNET GATEWAY & ROUTE ASSOCIATIONS

2.1 Subnet & Route Table Design

Rather than relying on the default main VPC route tables, three dedicated custom route tables were created and explicitly associated with their respective subnets:

1. **ewb-rt-a-public (rtb-0d500e5137b8f8b5e):** Explicitly associated with ewb-a-public (10.10.1.0/24). Contains a default route (0.0.0.0/0) pointing to the attached Internet Gateway (ewb-igw-a) so the Bastion host can accept incoming administrative SSH connections from the internet.
2. **ewb-rt-a-private (rtb-08f37f1e4dde757e8):** Explicitly associated with ewb-a-private (10.10.2.0/24). Contains no default internet route (0.0.0.0/0), ensuring ewb-app-a remains isolated from direct internet access.
3. **ewb-rt-b-private (rtb-00ec64c8de0a6ed09):** Explicitly associated with ewb-b-private (10.20.1.0/24). VPC B has no Internet Gateway attached, keeping ewb-app-b completely private.



Caption: Host: AWS VPC Console | Test: Route Table Associations | Result: Explicit subnet associations verified for ewb-rt-a-public, ewb-rt-a-private, and ewb-rt-b-private

03 / VPC PEERING, FORWARD PATH & RETURN PATH

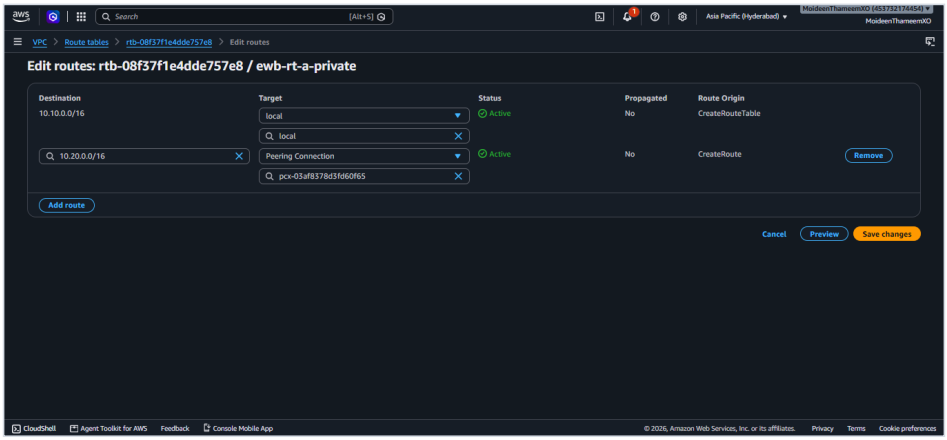
3.1 Peering Activation & Directional Routing

A same-account, same-region VPC peering request (ewb-a-b-peer / pcx-03af8378d3fd60f65) was initiated from Requester ewb-vpc-a (vpc-074a3adcf92fbc0f0) and accepted by Accepter ewb-vpc-b (vpc-0ac886432445110a9).

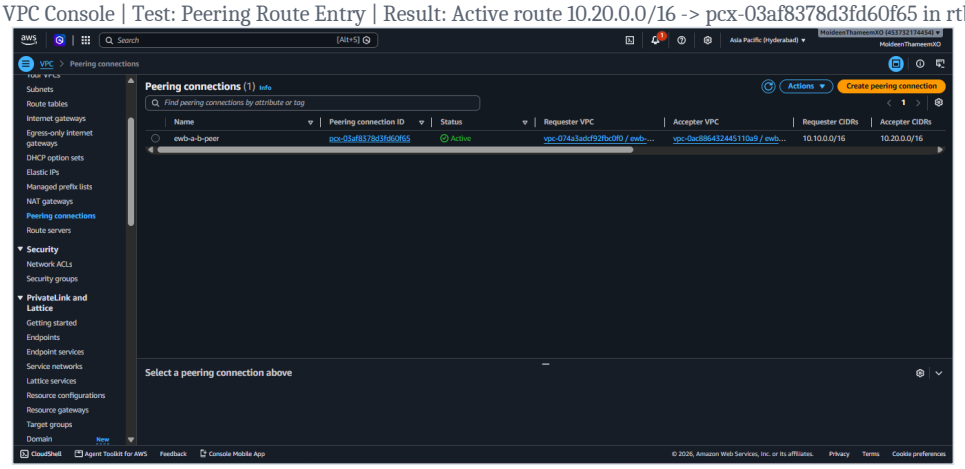
3.2 Forward Path vs. Return Path Explanation

- **Forward Path (App-A -> App-B):** When ewb-app-a (10.10.2.10) sends an ICMP or TCP packet to ewb-app-b (10.20.1.10), its subnet route table (ewb-rt-a-private) matches the destination 10.20.0.0/16 and forwards the packet across target pcx-03af8378d3fd60f65.
- **Return Path (App-B -> App-A):** Network routing is unidirectional; a one-way route only delivers the request, not the reply. For ewb-app-b to send ICMP Echo Replies or TCP ACKs back to VPC A, ewb-rt-b-private is configured with a return route for 10.10.0.0/16 targeting pcx-03af8378d3fd60f65.
- **Public Subnet Peering Route (Bastion -> App-B):** Because OpenSSH ProxyJump causes the Bastion host (10.10.1.10) to open a separate SSH connection directly to ewb-app-b (10.20.1.10), ewb-rt-a-public also includes the route 10.20.0.0/16 -> pcx-03af8378d3fd60f65. Using Longest Prefix Match (LPM), traffic to 10.20.0.0/16 takes the peering link rather than the less-specific 0.0.0.0/0 internet gateway route.

Route Table	Associated Subnet	Destination	Target	Purpose
ewb-rt-a-private	ewb-a-private	10.10.0.0/16	local	Intra-VPC A traffic
		10.20.0.0/16	pcx-03af8378d3fd60f65	Forward path: App-A to VPC B
ewb-rt-b-private	ewb-b-private	10.20.0.0/16	local	Intra-VPC B traffic
		10.10.0.0/16	pcx-03af8378d3fd60f65	Return path: VPC B to VPC A
ewb-rt-a-public	ewb-a-public	10.10.0.0/16	local	Bastion to App-A
		0.0.0.0/0	ewb-igw-a	Bastion internet access
		10.20.0.0/16	pcx-03af8378d3fd60f65	Bastion SSH jump to App-B



Caption: Host: AWS VPC Console | Test: Peering Route Entry | Result: Active route 10.20.0.0/16 -> pcx-03af8378d3fd60f65 in rtb-08f37f1e4dde757e8



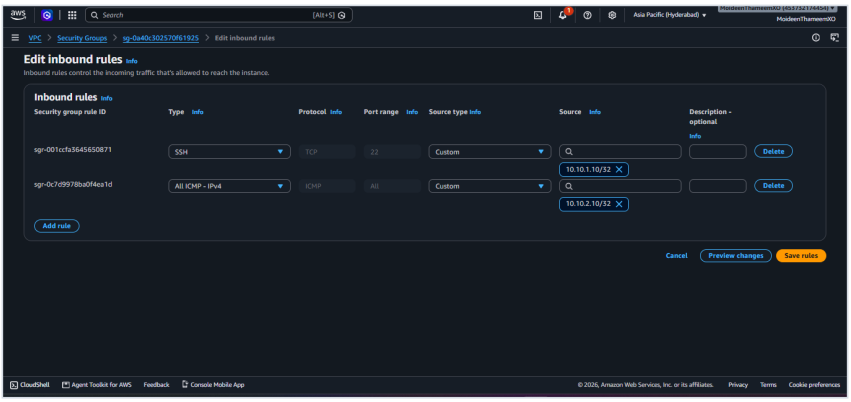
Caption: Host: AWS VPC Console | Test: Peering Status | Result: ewb-a-b-peer (pcx-03af8378d3fd60f65) is Active

04 / SECURITY RULES & PRIVATE SSH ACCESS

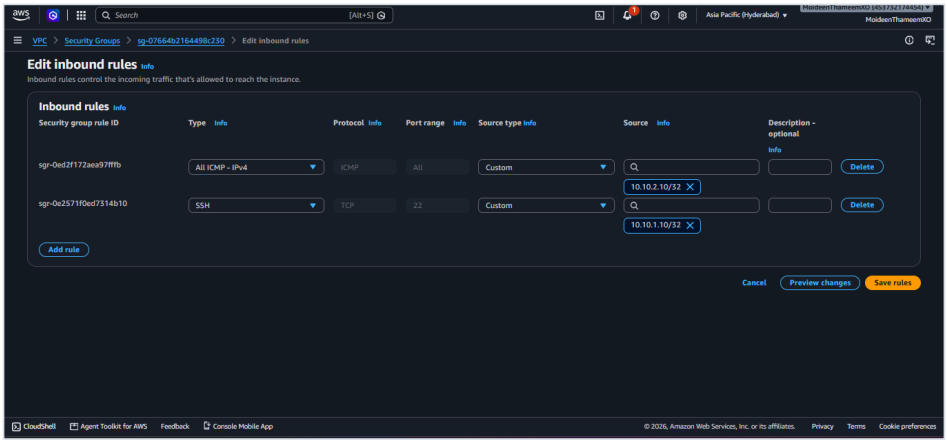
4.1 Principle of Least Privilege (/32 Host-Scoped Rules)

Inbound security group rules are restricted to /32 single-host IPv4 addresses rather than broad subnet masks or 0.0.0.0/0:

- **ewb-sg-bastion:** Permits inbound SSH (TCP 22) only from the administrator workstation's public IP (49.37.171.106/32).
- **ewb-sg-app-a (sg-0a40c302570f61925):** Permits inbound SSH (TCP 22) strictly from the Bastion's private IP (10.10.1.10/32) and inbound ICMP from the peer workload.
- **ewb-sg-app-b (sg-07664b2164498c230):** Permits inbound SSH (TCP 22) strictly from the Bastion's private IP (10.10.1.10/32) and inbound ICMP from ewb-app-a (10.10.2.10/32).



Caption: Host: AWS VPC Console | Test: App-A Security Group (sg-0a40c302570f61925) | Result: Scoped /32 SSH and ICMP rules configured



Caption: Host: AWS VPC Console | Test: App-B Security Group (sg-07664b2164498c230) | Result: Inbound SSH allowed from 10.10.1.10/32 and ICMP allowed from 10.10.2.10/32

4.2 Local SSH ProxyJump Configuration (~/.ssh/config)

To connect to ewb-app-a and ewb-app-b without copying the private .pem key onto the Bastion host, ProxyJump was configured on the local workstation after securing NTFS file permissions (icacls):

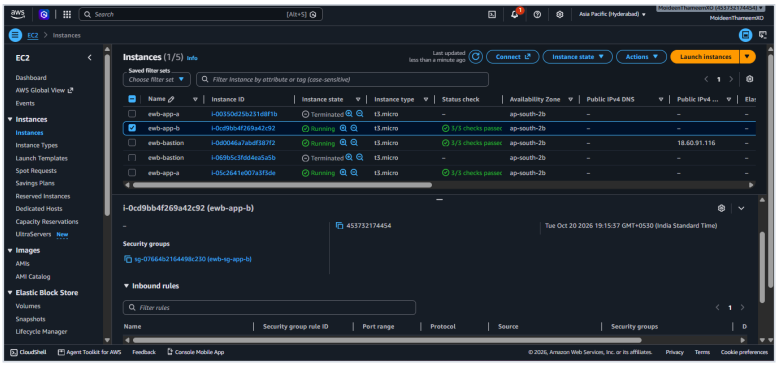
```
Host ewb-bastion
    HostName 18.60.91.116
    User ec2-user
    IdentityFile C:\Users\hamza\Keys\ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile C:\Users\hamza\Keys\ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
    IdentityFile C:\Users\hamza\Keys\ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

4.3 Test 05: Workload Isolation Verification

Inspection of the EC2 console confirms that ewb-bastion is assigned public IPv4 18.60.91.116, while both ewb-app-a (i-05c2641e007a3f3de) and ewb-app-b (i-0cd9bb4f269a42c92) have empty Public IPv4 address fields.

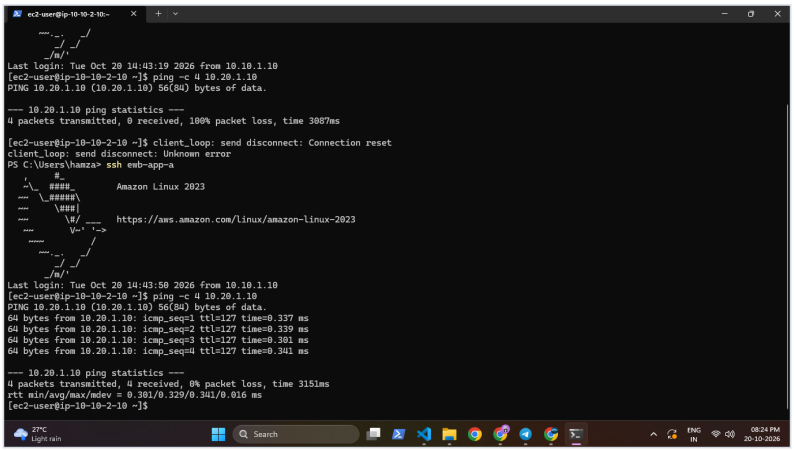


Caption: Host: AWS EC2 Console | Test 05: Workload Isolation | Result: ewb-app-a and ewb-app-b have no Public IPv4 addresses

05 / POSITIVE & NEGATIVE TEST EVIDENCE

5.1 Test 01: Baseline Failure (Before Peering & Routes)

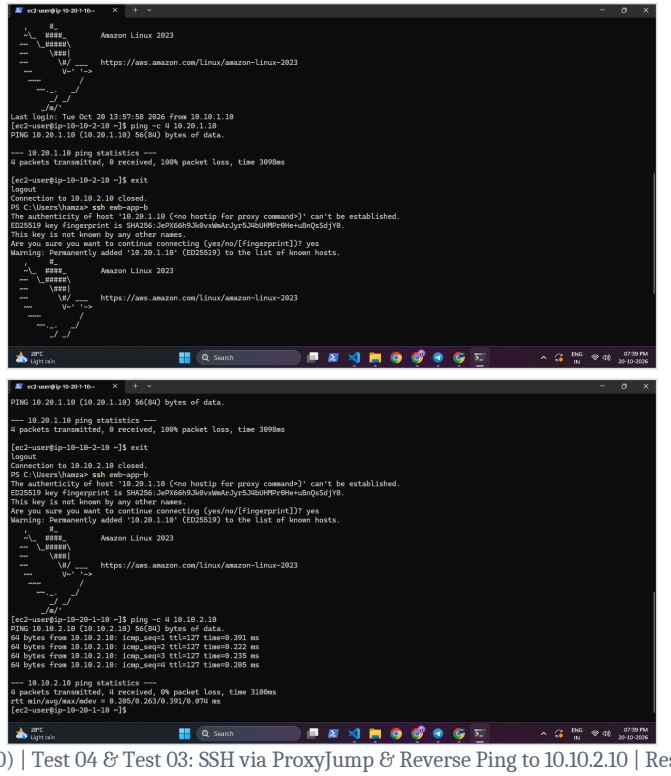
Prior to creating the peering connection and updating the route tables, an SSH session was established to ewb-app-a (10.10.2.10) via the Bastion host, and ping -c 4 10.20.1.10 was executed. As expected, all 4 packets were lost (100% packet loss) because VPC A had no route to 10.20.0.0/16.



Caption: Host: ewb-app-a (10.10.2.10) | Test 01: Baseline Ping to 10.20.1.10 Before Peering | Result: 4 packets transmitted, 0 received, 100% packet loss

5.2 Test 04 & Test 03: Private SSH to App-B & Reverse Path Ping Success

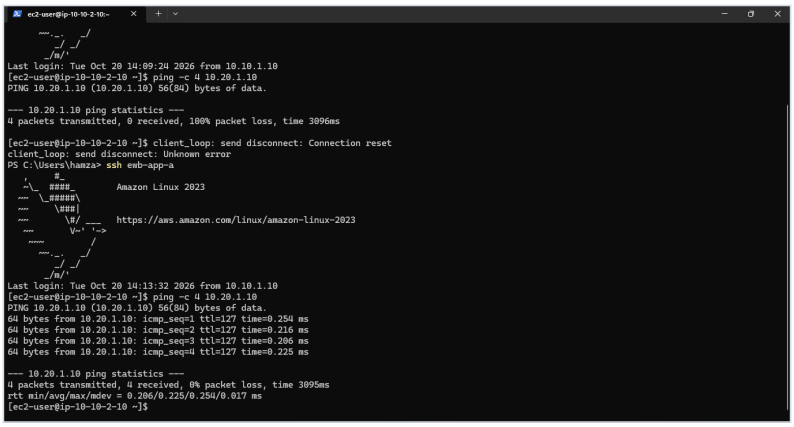
After activating pcx-03af8378d3fd60f65 and configuring the three route tables, ssh ewb-app-b was executed from the laptop. The session jumped cleanly through ewb-bastion into ip-10-20-1-10. Running ping -c 4 10.10.2.10 verified the reverse path back to ewb-app-a with 0% packet loss (rtt avg = 0.263 ms).



Caption: Host: ewb-app-b (10.20.1.10) | Test 04 & Test 03: SSH via ProxyJump & Reverse Ping to 10.10.2.10 | Result: SSH login succeeded; 4 packets received, 0% packet loss

5.3 Test 02: Forward Path Ping Success (App-A to App-B)

From ewb-app-a (10.10.2.10), ping -c 4 10.20.1.10 was executed across the peering link. All 4 packets returned replies from 10.20.1.10 with 0% packet loss (rtt avg = 0.225 ms).

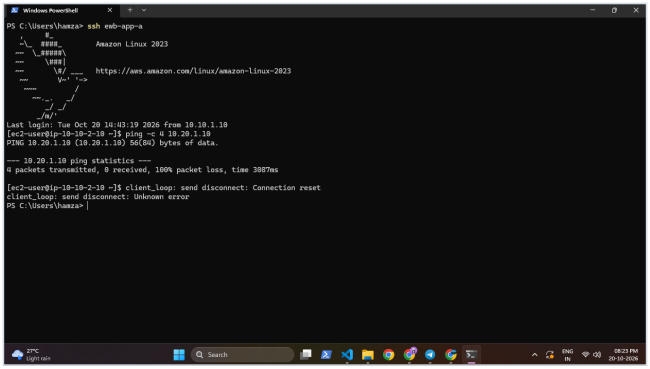


Caption: Host: ewb-app-a (10.10.2.10) | Test 02: Forward Ping to 10.20.1.10 After Peering | Result: 4 packets transmitted, 4 received, 0% packet loss

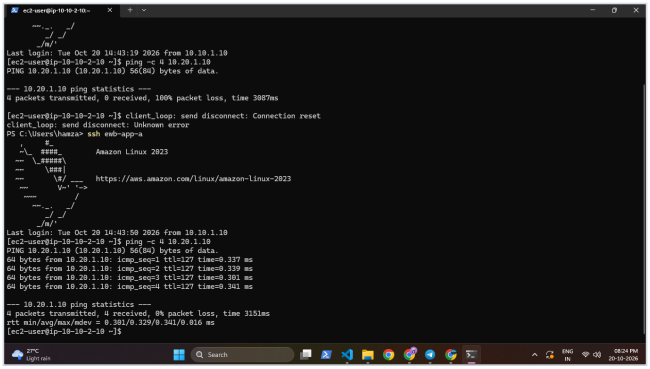
5.4 Controlled ICMP Failure & Recovery

To prove that security groups actively enforce traffic filtering across an active peering connection, a controlled failure test was performed:

1. **Controlled Failure:** The inbound ICMP rule allowing 10.10.2.10/32 on ewb-sg-app-b was temporarily removed while leaving the SSH rule and route tables untouched. Re-running ping -c 4 10.20.1.10 from ewb-app-a immediately resulted in 100% packet loss.
2. **Controlled Recovery:** Restoring the inbound ICMP rule on ewb-sg-app-b for source 10.10.2.10/32 immediately restored connectivity, with ping -c 4 10.20.1.10 succeeding with 0% packet loss (rtt avg = 0.329 ms).



Caption: Host: ewb-app-a (10.10.2.10) | Test: Controlled ICMP Failure | Result: 100% packet loss when inbound ICMP rule is removed from ewb-sg-app-b



Caption: Host: ewb-app-a (10.10.2.10) | Test: Controlled ICMP Recovery | Result: 0% packet loss immediately after restoring the inbound ICMP rule on ewb-sg-app-b

06 / VIVA EXPLANATIONS, REFLECTION & CLEANUP

6.1 Viva Technical Questions

1. Why is the return route needed?

Routing in AWS is directional and non-symmetric. While the forward route in ewb-rt-a-private delivers packets from VPC A to VPC B across pcx-03af8378d3fd60f65, ewb-app-b cannot send ICMP Echo Replies or TCP ACK packets back unless ewb-rt-b-private explicitly routes 10.10.0.0/16 back to the peering connection.

2. Why is the App-B SSH source the bastion private IP?

When connecting via OpenSSH ProxyJump, the laptop authenticates to ewb-bastion first, and the Bastion host opens a second TCP connection from its own private network interface (10.10.1.10) across the VPC peering link to ewb-app-b (10.20.1.10). Therefore, ewb-app-b sees 10.10.1.10 as the source IP.

3. Why does Active peering not guarantee a successful ping?

An Active peering connection only creates the virtual link between the VPCs. Traffic still cannot flow unless: (a) both VPC route tables have routes pointing each other's CIDR blocks to the pcx-ID, and (b) the instance Security Groups (and NACLs) allow the outbound request and inbound protocol traffic.

4. Can VPC B use VPC A's internet gateway over peering?

No. AWS VPC Peering is strictly one-to-one and non-transitive, and it does not support edge-to-edge routing. Instances in VPC B (ewb-app-b) cannot route internet-bound traffic (0.0.0.0/0) through ewb-igw-a in VPC A.

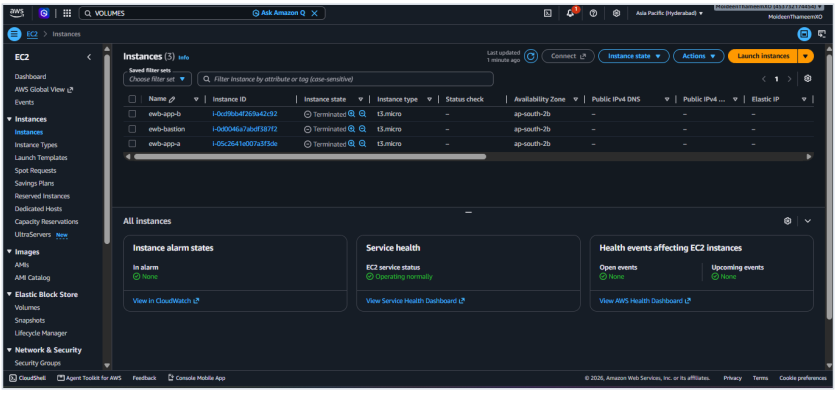
6.2 Troubleshooting & Learning Reflection

- **Local SSH Key Permissions on Windows:** OpenSSH initially rejected ewb-lab.pem with WARNING: UNPROTECTED PRIVATE KEY FILE! because inherited Windows NTFS permissions granted read access to other system principals. Using icacls to strip inheritance (/inheritance:r) and grant exclusive read/write access to the local user resolved the authentication error.
- **Asymmetric Ping Due to Missing Outbound Security Group Rule:** After activating peering, reverse ping (App-B -> App-A) succeeded with 0% packet loss, while forward ping (App-A -> App-B) failed with 100% packet loss. Because reverse ping worked, both route tables and the peering link were proven healthy. Inspecting the security groups revealed that ewb-sg-app-a was missing its default outbound allow rule (0.0.0.0/0), preventing ewb-app-a from transmitting the initial Echo Request while still allowing stateful replies when ewb-app-b initiated the ping. Restoring the outbound allow rule immediately fixed forward connectivity.

6.3 Resource Teardown & Cleanup Verification

To prevent ongoing AWS billing charges, all lab resources were deleted in reverse dependency order:

1. Terminated EC2 instances ewb-bastion (i-0d0046a7abdf387f2), ewb-app-a (i-05c2641e007a3f3de), and ewb-app-b (i-0cd9bb4f269a42c92), confirming root EBS volumes were automatically deleted on termination.
2. Deleted the VPC Peering connection ewb-a-b-peer (pcx-03af8378d3fd60f65) and peer route entries.
3. Deleted custom security groups (ewb-sg-bastion, ewb-sg-app-a, ewb-sg-app-b), subnets (ewb-a-public, ewb-a-private, ewb-b-private), and custom route tables (ewb-rt-a-public, ewb-rt-a-private, ewb-rt-b-private).
4. Detached and deleted Internet Gateway ewb-igw-a, then deleted ewb-vpc-a (vpc-074a3adcf92fbc0f0) and ewb-vpc-b (vpc-0ac886432445110a9).



Caption: Host: AWS Console (ap-south-2) | Test: Resource Cleanup | Result: All 3 EC2 instances terminated and custom VPC resources deleted